Jaya Stores E-commerce Platform Documentation

Author: Kishore Balaji P
Date: July 2025

Contents

Table of Contents

Abstract / Executive Summary

Jaya Stores is a full-stack online retail platform developed to serve the e-commerce needs of my village. It features a React/TypeScript frontend hosted on Vercel and a Node.js/Express backend with a PostgreSQL (Neon DB) database, while product images are hosted on AWS S3. The backend, which serves the application APIs, is deployed on AWS Lambda (serverless) for scalability. Key features include product browsing, user registration with OTP verification, shopping cart, Payment gateway integration attempted (currently inactive due to account verification issues) Cash on Delivery (COD) as an additional payment option, and a chat bot service to assist users. The platform is designed for local users and is expected to generate a profit of approximately ₹50,000.

Objectives and Scope

- **User Shopping Experience:** Provide a seamless online shopping experience with features such as account registration, product search and browsing, shopping cart management, and secure checkout with Razorpay payment processing (test/sandbox mode) and Cash on Delivery (COD) as an additional payment option. User registration includes accepting terms and conditions, after which an OTP is sent for verification. Only after verifying the OTP can the user complete registration and log in.
- **Admin Management Tools:** Enable store administrators to manage the platform efficiently through a comprehensive dashboard that displays number of users, orders, products, and revenue. Admins can perform order management, product management, and user management, as well as add or edit products, manage inventory, and view sales reports.
- **Modern Tech Stack:** Use a modern, maintainable technology stack. The frontend is built with React for interactive UIs and Tailwind CSS for utility-first styling, while the backend uses Node.js (event-driven, non-blocking runtime) with Express for API services. PostgreSQL is used as the database.
- **Scalability & Reliability:** Architect the system to handle growing traffic. The backend, built on Node.js, supports asynchronous, non-blocking operations

for scalable network applications. APIs are hosted on AWS Lambda (serverless functions), allowing automatic scaling without manual server management.

- **Security & Compliance:** Implement standard security measures such as hashed passwords, JWT authentication, and HTTPS for all API calls.

The project scope includes user-facing features such as product browsing, shopping, checkout, and viewing order history, as well as administrative features including product, order, and user management through a comprehensive dashboard. The development process incorporates continuous integration and automated testing to maintain code quality, and the platform is deployed on AWS cloud services for scalability and reliability.

Tech Stack (Frontend & Backend)

The project uses a modern and maintainable technology stack. The frontend is built with React (TypeScript) for interactive and type-safe UIs, Tailwind CSS for utility-first styling, Vite as a fast build tool with hot-module reload, Axios for communicating with backend APIs, and React Router for smooth page navigation. The backend uses Node.js for its event-driven, non-blocking architecture and Express to build RESTful APIs. Data is stored in PostgreSQL (Neon DB), while product images are hosted on AWS S3. The backend runs on AWS Lambda (serverless) for scalability, with deployment automated via GitHub Actions. Razorpay API is integrated for secure payment processing (currently in sandbox mode).

Frontend Description:

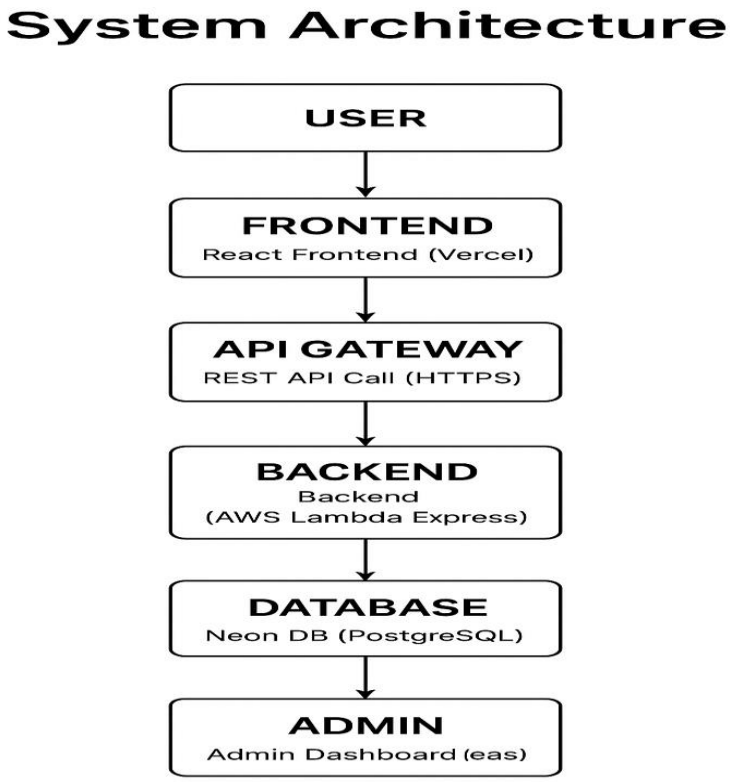
- React is a component-based library for building interactive UIs, and TypeScript adds static typing for safer, more maintainable code.
- Tailwind CSS provides utility classes for rapid styling.
- Vite is used as the build tool for fast development with hot-module reload.
- Axios enables communication with backend APIs for data fetching.

- React Router allows smooth page navigation and routing between different sections of the application.

Backend Description:

- Node.js is chosen for its non-blocking, event-driven architecture, ideal for scalable network applications.
- Express provides a framework for building RESTful APIs.
- PostgreSQL is used for its robustness and ability to handle complex data workloads.
- AWS S3 stores product images, while AWS Lambda hosts the backend in a serverless, scalable environment, with deployments automated via GitHub Actions
- Phonepe Payment gateway integration attempted (currently inactive due to account verification issues)

System Architecture



Example architecture of Jaya Stores.

The Jaya Stores application follows a modular, cloud-native design. Users interact with the platform via a browser or mobile device, which communicates with the frontend (React/TypeScript hosted on Vercel). The frontend sends requests to the backend REST API (Node.js/Express) to handle business logic and data operations.

The backend persists application state in PostgreSQL (Neon DB), while product images are stored on AWS S3 for durability and high availability. A chat bot service assists users with queries and navigation.

This architecture ensures a clear separation of concerns, with the frontend handling presentation, the backend managing logic and data, and external services providing image storage, chat assistance, and payment processing.

Features Overview

User-Facing Features:

- Account Management: Users can register, log in, and manage their profiles. Authentication is secured with JWT-based token sessions, stored in localStorage to maintain login across browser refreshes. Registration includes OTP verification for added security.
- Product Catalog: Users can browse and search products by category or keyword. Each product page displays images, descriptions, prices, and available stock.
- Shopping Cart: Users can add or remove items in a personal cart, which persists on the backend for logged-in accounts.
- Checkout & Payments: Payment gateway integration attempted (currently inactive due to account verification issues). Cash on Delivery (COD) is also supported.
- Order History: Registered users can view past orders and their status. Email notifications are sent on order confirmation and shipping.
- Chat Bot Service: A chatbot assists users in navigating the store, finding products, and answering queries.

- Responsive Design: The UI, built with Tailwind CSS, is fully responsive for desktop and mobile browsers.

Admin Panel Features:

- Authentication: Secure admin login separate from customer accounts.
- Product Management: Administrators can create, read, update, and delete products. Product details include titles, descriptions, images, prices, categories, variants, and inventory levels. Bulk import via CSV is supported.
- Order Management: Admins can view all orders, update statuses (e.g., Pending, Shipped, Delivered), and access customer details.
- Customer Management: Admins can view registered users and their order histories, with optional features for searching or deactivating accounts.
- Analytics Dashboard: Key metrics, including total sales, orders, and product counts, are aggregated. While the current version logs basic statistics, future versions may include visual charts and graphs.

Most features follow standard e-commerce conventions, such as account creation, order history, and shopping cart for users, and full CRUD operations for products, order management, and analytics in the admin panel. The use of localStorage improves the user experience by persisting cart items for guest users.

Folder Structure

Frontend

```
├── public/                 # Static assets (images, favicon, etc.)
├── src/
|   ├── App.tsx             # Main app component. Sets up all routes and context providers
|   ├── main.tsx            # Entry point. Renders the App into the DOM
|   ├── index.css           # Tailwind CSS imports and global styles
|   ├── components/         # Reusable UI components
|   |   ├── Cart/
|   |   |   └── CartItem.tsx      # UI and logic for a single cart item
|   |   ├── Chatbot/
|   |   |   └── Chatbot.tsx       # Interactive customer support
```

```
|   |   ├── Common/
|   |   |   └── ProtectedRoute.tsx  # Route protection by auth/role
|   |   ├── Layout/
|   |   |   ├── AdminSidebar.tsx    # Admin navigation sidebar
|   |   |   ├── Footer.tsx          # Shared footer component
|   |   |   └── Navbar.tsx          # Main navigation header
|   |   └── Products/
|   |       └── ProductGrid.tsx     # Grid layout for products
|   ├── context/
|   |   ├── AuthContext.tsx         # Authentication state management
|   |   └── CartContext.tsx         # Shopping cart state
|   ├── pages/
|   |   ├── Admin/
|   |   |   ├── Dashboard.tsx       # Admin statistics
|   |   |   ├── OrdersManagement.tsx
|   |   |   ├── ProductsManagement.tsx
|   |   |   └── UsersManagement.tsx
|   |   ├── Auth/
|   |   |   ├── ForgotPassword.tsx
|   |   |   ├── Login.tsx
|   |   |   ├── Register.tsx
|   |   |   └── VerifyRegistrationOtp.tsx
|   |   ├── Legal/
|   |   |   ├── ContactUs.tsx
|   |   |   ├── PrivacyPolicy.tsx
|   |   |   ├── RefundPolicy.tsx
|   |   |   ├── ShippingPolicy.tsx
|   |   |   └── TermsAndConditions.tsx
|   |   └── User/
|   |       ├── PaymentCallback.tsx
|   |       ├── Cart.tsx
|   |       ├── Checkout.tsx
|   |       ├── Home.tsx
|   |       ├── OrderConfirmation.tsx
|   |       ├── Orders.tsx
|   |       ├── ProductDetails.tsx
```

```
|   |       └── Profile.tsx
|   ├── services/
|   |   └── api.ts               # API integration with Axios
|   └── types/
|       └── index.ts             # TypeScript interfaces
├── .env                         # Environment variables
├── eslint.config.js             # ESLint configuration
├── postcss.config.js            # PostCSS setup
├── tailwind.config.js           # Tailwind CSS configuration
├── tsconfig.json                # TypeScript configuration
├── vite.config.ts               # Vite configuration
└── index.html                   # HTML entry point
```

Backend

```
├── app.js           # Main application setup with serverless handler
├── db.js            # Database connection (PostgreSQL)
├── migrations.sql   # SQL schema for users, products, cart, orders, etc.
├── package.json     # Project metadata and dependencies
├── .github/         # GitHub configuration
|   └── workflows/
|       └── deploy-lambda.yml  # AWS Lambda deployment workflow
├── middleware/      # Custom Express middlewares
|   ├── admin.js     # Admin role check middleware
|   └── auth.js      # JWT authentication middleware
├── routes/          # API route handlers
|   ├── admin.js     # Admin-only endpoints (products, orders, users)
|   ├── auth.js      # User registration and login
|   ├── cart.js      # Cart management endpoints
|   ├── chatbot.js   # Chatbot message endpoint (Gemini API)
|   ├── orders.js    # Order creation and retrieval
|   ├── payments.js  # Payment integration (phonepe)
|   ├── products.js  # Product listing and details
|   ├── otpAuth.js   # OTP-based password reset endpoints
|   └── user.js      # User shipping address management
└── utils/           # Utility functions
    ├── email.js     # Email sender (order confirmations, OTP)
```

```
├── s3.js        # AWS S3 integration for image uploads
└── s3-test.js   # S3 integration testing utilities
```

This structure separates concerns: the frontend src/ contains all React code, organized by components and pages, while the backend src/ contains Express controllers, models (database schemas), and routes. Configuration details such as database URIs, API keys, and other environment-specific settings are stored in the root .env file, which is not committed to source control. This ensures sensitive information remains secure while keeping the project organized and maintainable.

Key API Endpoints

All backend functionality is exposed via RESTful API endpoints. Key endpoints include (authenticated routes use JWT tokens):

Route (URL)	Method	Description
`POST /api/auth/register`	POST	Create a new user account (register).
`POST /api/auth/login`	POST	Authenticate user and return token.
`GET /api/products`	GET	List all products (with optional query filters).
`GET /api/products/:id`	GET	Get details for a single product by ID.
`POST /api/cart`	POST	Add an item to the current user's cart (or update qty).
`GET /api/cart`	GET	Retrieve the current user's cart contents.
`DELETE /api/cart/:itemId`	DELETE	Remove an item from the cart by item ID.
`POST /api/checkout`	POST	Process checkout via COD.
`POST /api/orders`	POST	Create a new order (called during checkout).
`GET /api/orders`	GET	Get all orders for the current user (order history).
`GET /api/orders/:id`	GET	Get details of a specific order (if allowed).
`GET /api/admin/products`	GET	(Admin) List all products.
`POST /api/admin/products`	POST	(Admin) Add a new product.
`PUT /api/admin/products/:id`	PUT	(Admin) Update an existing product.
`DELETE /api/admin/products/:id`	DELETE	(Admin) Delete a product.
`GET /api/admin/orders`	GET	(Admin) List all orders in the system.
`PUT /api/admin/orders/:id`	PUT	(Admin) Update order status or details.

These routes follow standard REST conventions, using plural nouns and HTTP verbs to indicate actions, as recommended for web APIs. Routes that require authentication expect a valid JWT in the request header. The API consistently

returns JSON responses, and error-handling middleware catches exceptions, returning appropriate HTTP status codes.

Deployment Overview

- Backend (AWS Lambda): The Node.js/Express backend is deployed as individual AWS Lambda functions. AWS API Gateway exposes REST endpoints publicly. This serverless model ensures automatic scaling without managing servers. Logs and metrics are monitored via AWS CloudWatch. Deployment is automated using GitHub Actions, so pushing code to GitHub triggers a pipeline that updates the Lambda functions. All sensitive environment variables (API keys, database connection strings) are stored securely within Lambda.
- Database (Neon DB): The backend uses Neon DB, a serverless PostgreSQL database, for storing users, products, orders, and cart data. Neon provides scalability, automatic backups, and a fully managed environment, simplifying database maintenance.
- Frontend Hosting: The React frontend is deployed on Vercel, which serves the application and handles routing, while AWS S3 is used only for product images. This separation ensures fast, reliable delivery of static assets and application code.
- Logging & Monitoring: AWS CloudWatch tracks backend logs and metrics.

This deployment approach ensures a scalable, reliable, and cost-effective e-commerce platform: AWS Lambda handles backend logic, Neon DB provides a serverless database, Vercel hosts the frontend efficiently, and S3 stores product images. GitHub Actions automates CI/CD to deploy the latest tested code.

Personal Contributions

- As the sole developer of Jaya Stores, I implemented the entire system, including backend, frontend, deployment, and integrations:
- Backend Development: Designed and coded the Node.js + Express API. Created the database schema (users, products, orders, carts) and implemented all API endpoints (authentication with OTP, products, cart, orders). Used Neon DB as the PostgreSQL database. Implemented JWT-based authentication, input validation, and error handling.
- Frontend Development: Built the React/TypeScript application with Tailwind CSS. Developed reusable UI components (e.g., ProductCard, CartItem), pages (Home, Shop, ProductDetail, Cart, Checkout), and routing. Ensured responsive design for desktop and mobile. Configured Vite for fast development builds and optimized production bundling.
- AWS & Hosting Deployment: Configured AWS Lambda for the backend and attached API Gateway to expose the REST endpoints. Used AWS S3 for product image storage. Deployed the frontend to Vercel, ensuring fast and reliable delivery. Managed environment variables and secrets securely in Lambda.
- Payment Integration: Payment gateway integration attempted (currently inactive due to account verification issues)
- DevOps & Testing: Set up a CI/CD pipeline using GitHub Actions to run tests and deploy backend updates automatically to Lambda. Wrote unit and integration tests for both backend APIs and frontend components to ensure reliability.

Challenges Faced and Solutions

- API Authentication & Security: Ensuring secure user authentication was critical. Implemented JWT tokens with short expiry and secure password hashing (bcrypt). CORS issues between the Vercel frontend and Lambda backend required proper header configuration in Express.

- Asynchronous Programming: Handling asynchronous operations (DB queries, cart updates, payment callbacks) required careful use of async/await and centralized error-handling middleware to avoid callback hell.
- AWS Deployment Complexity: Packaging Express as Lambda functions required adapting the code (using serverless-http). Initial cold-start delays were mitigated by minimizing bundle size and keeping functions small. Setting up CORS and SSL on API Gateway also required effort.
- Tailwind CSS Learning Curve: Designing components without a pre-existing style guide required iteration. Utility classes and a customized tailwind.config.js helped maintain consistency and brand colors.

Future Enhancements

- Implement Live Phonepe Payment gateway integration
- Customer Reviews & Ratings: Allow users to leave product reviews and ratings, with moderation tools for admins.
- Performance Optimization: Introduce caching (e.g., Redis) for frequently accessed data to reduce DB load and improve response time.
- Analytics & Reporting: Enhance the admin dashboard with charts/graphs for sales trends, customer activity, and order statistics.
- Mobile App / PWA: Develop a Progressive Web App or native mobile app to reach more customers.
- Internationalization: Add multi-language support and multi-currency pricing for broader reach.

These enhancements would improve platform functionality, user experience, and scalability. Implementing Phonepe to Live mode and adding advanced features would be key growth areas.

Conclusion

Jaya Stores is a complete e-commerce solution built with modern web technologies and cloud infrastructure. It demonstrates a full user journey—from browsing products to secure checkout—along with an administrative backend for managing the store.

The architecture leverages serverless AWS Lambda functions for the backend, Neon DB for a scalable serverless PostgreSQL database, Vercel for frontend hosting, and S3 for product images, ensuring scalability and cost-efficiency.

The frontend stack (React, TypeScript, Tailwind CSS, Vite) provides a fast and responsive user interface and a chat bot assists users.

As the sole developer, I implemented all core features, set up deployments, handled integrations, and resolved key technical challenges. Overall, Jaya Stores meets its objectives of providing a robust, maintainable e-commerce platform and lays a strong foundation for future growth.

Sources

1. Global Ecommerce Sales Growth Report (2025) - Shopify
   https://www.shopify.com/blog/global-ecommerce-sales
2. React | Meta Open Source
   https://opensource.fb.com/projects/react/
3. Tailwind CSS - Rapidly build modern websites
   https://tailwindcss.com/
4. Node.js Introduction | The Odin Project
   https://www.theodinproject.com/lessons/nodejs-introduction-what-is-nodejs
5. Vite Guide
   https://vite.dev/guide/

6. AWS Guidance for Web Store
https://d1.awsstatic.com/architecture-diagrams/ArchitectureDiagrams/guidance-for-web-store-on-aws-ra.pdf
7. PostgreSQL: About
https://www.postgresql.org/about/
8. Top E-commerce Website Features – Admin Panel
https://seclgroup.com/top-ecommerce-website-features-part-2/
9. Web API Design Best Practices – Microsoft Learn
https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design
10. A Guide to Developing Serverless E-commerce Workflows | AWS
https://aws.amazon.com/blogs/industries/a-guide-to-developing-serverless-ecommerce-workflows-for-commercetools/